Guia de Variáveis de Ambiente — Agenda de Boteco

Recriar do zero todas as variáveis de ambiente e segredos: Supabase, PostHog, Cloudflare R2, OTA, Vercel, EAS e GitHub Actions. Versão de 2026-11-09 (PR #115).

ATENÇÃO — segredos

Nunca cole chave, token ou senha em arquivo do repo, em `.env.example`, em issue ou em chat. Valores reais vivem só no gerenciador de senhas, nos painéis (Supabase, PostHog, Cloudflare, Vercel, EAS, GitHub) e no seu terminal durante a configuração. A `SUPABASE_SERVICE_ROLE_KEY` nunca entra em app cliente (Vercel ou EAS).

Sobre os botões de copiar

Os botões funcionam na versão HTML deste guia (`docs/setup/guia-variaveis-ambiente.html`). No PDF, selecione o texto do bloco e copie.

Ordem de execução

1. Preparar ferramentas e o terminal
2. Coletar valores nas origens (Supabase, PostHog, R2, Google, Expo, Apple)
3. Gerar o par de chaves de assinatura do OTA
4. Carregar os valores no terminal
5. Supabase — segredos das Edge Functions
6. GitHub Actions — secrets e variables
7. Vercel — 5 projetos web
8. EAS — 2 apps mobile
9. Desenvolvimento local (`.env.local`)
10. Verificar tudo
11. Provisionar o PostHog (script)
12. Apêndice — inventário completo

---

1. Preparar ferramentas e o terminal

1. Confirme que as CLIs estão instaladas (todas já existem nesta máquina):

```
gh --version && eas --version && supabase --version && vercel --version
```

2. Faça login em cada uma:

```
gh auth login
eas login
supabase login
vercel login
```

3. Abra um terminal novo só para esta configuração e desligue o histórico dele, para que nenhuma chave fique salva em `~/.zsh_history`:

```
unset HISTFILE
cd ~/git/agenda-de-boteco
```

> **CUIDADO**
> Use sempre este mesmo terminal até o fim do passo 10. Ao fechar, as variáveis carregadas somem.

---

2. Coletar valores nas origens

Anote cada valor no gerenciador de senhas. Os nomes da coluna "Variável no terminal" são usados no passo 4.

2.1 Supabase (projeto de produção)

1. Abra `https://supabase.com/dashboard` e selecione o projeto.
2. **Project Settings → General**: copie o *Reference ID*.
3. **Project Settings → Data API** (ou *API*): copie a *Project URL*.
4. **Project Settings → API Keys**: copie a chave *anon / public* (ou a *publishable*). Copie também a *service_role* (ou *secret*).
5. **Project Settings → Database**: anote a senha do banco. Se não souber, clique em *Reset database password*.
6. **Account → Access Tokens** (`https://supabase.com/dashboard/account/tokens`): crie um token chamado `github-actions-deploy` .

Valor	Variável no terminal	Sensível?
Reference ID	`SB_REF`	não
Project URL (`https://<ref>.supabase.co`)	`SB_URL`	não
anon / publishable key	`SB_ANON`	não (pública por design; a proteção é a RLS)
service_role / secret key	`SB_SERVICE`	SECRET
Senha do banco	`SB_DB_PASSWORD`	SECRET
Access token pessoal	`SB_ACCESS_TOKEN`	SECRET

2.2 PostHog (5 projetos, um por dupla de apps)

Projeto	Apps	Project ID	Project key no terminal
Consumidor	web + mobile	`591092`	`PH_KEY_CONSUMIDOR`

Dono do bar	web-client + mobile-client	`591110`	`PH_KEY_DONO`
Artistas	web-artists	`591111`	`PH_KEY_ARTISTAS`
Administrador	admin	`591113`	`PH_KEY_ADMIN`
Landing	landing	`591114`	`PH_KEY_LANDING`

1. Em `https://us.posthog.com`, abra cada projeto. Em **Settings → Project → General**, copie a *Project API key* (começa com `phc_`). Ela é pública por design.
2. Confirme a região. US: `https://us.i.posthog.com` (ingestão) e `https://us.posthog.com` (API). UE: troque `us` por `eu`.
3. Crie a chave de **upload**: avatar → **Settings → Personal API keys → Create**. Nome `agenda-build-upload`. Escopo nos 5 projetos. Permissões de escrita de Error tracking / source maps / symbol sets e leitura de project. Variável: `PH_UPLOAD_KEY` SECRET.
4. Crie a chave de **provisionamento**: nome `agenda-setup-script`, escopo nos 5 projetos. Permissões read e write em `insight`, `dashboard`, `alert`, `hog_function` e `project`. Variável: `PH_SETUP_KEY` SECRET. Ela é usada só no passo 11.

> **CUIDADO**
> A chave pessoal aparece uma única vez. Se o primeiro upload falhar com 403, edite a chave e adicione o escopo que a mensagem de erro citar.

2.3 Cloudflare R2 (bucket do OTA)

1. Em `https://dash.cloudflare.com`, copie o *Account ID* (barra lateral da conta). Variável: `R2_ACCOUNT`.
2. **R2 → Create bucket**: nome `agenda-ota`. Se já existir, mantenha.
3. No bucket: **Settings → Public access**. Conecte um domínio próprio (recomendado) ou habilite a URL `r2.dev`. Copie a URL pública, sem barra no fim. Variável: `R2_PUBLIC_URL`.
4. **R2 → Manage API tokens → Create API token**: permissão *Object Read & Write*, restrita ao bucket `agenda-ota`. Copie o *Access Key ID* (`R2_KEY_ID`) e o *Secret Access Key* (`R2_SECRET` SECRET).

2.4 Google Maps (só app mobile do consumidor)

1. Em `https://console.cloud.google.com`: **APIs & Services → Credentials → Create credentials → API key**.
2. Crie uma chave para iOS (restrição: *iOS apps*, bundle do app) e uma para Android (restrição: *Android apps*, pacote + SHA-1 do certificado de upload).
3. Habilite *Maps SDK for iOS* e *Maps SDK for Android*.
4. Variáveis: `GMAPS_IOS` e `GMAPS_ANDROID`.

2.5 Expo / EAS

1. Em `https://expo.dev`: **Account settings → Access tokens → Create token**, nome `github-actions`. Variável: `EXPO_TOKEN_VALUE` SECRET.

2. Os IDs de projeto EAS já ficam fixos no código: `apps/mobile/app.config.ts` (`14d06c33-…`) e `apps/mobile-client/app.config.ts` (`05db6b05-…`). Se um projeto EAS for recriado, troque o `extra.eas.projectId` no arquivo e commite.

2.6 App Store Connect

1. Em `https://appstoreconnect.apple.com` → **Apps**: abra cada app → **App Information** → copie o *Apple ID* numérico.
2. Variáveis: `ASC_ID_CONSUMIDOR` (app consumidor) e `ASC_ID_DONO` (app do dono do bar).

2.7 URLs públicas

Variável no terminal	Valor esperado
`SITE_URL`	Domínio da landing, ex. `https://agendadeboteco.com`
`WEB_URL`	`$SITE_URL/app` (web pública do consumidor)
`OTA_MANIFEST_URL`	`$SB_URL/functions/v1/ota-manifest`

---

3. Gerar o par de chaves de assinatura do OTA

> **ATENÇÃO — chave nova invalida os binários instalados**
> O `certificate.pem` fica embutido no binário dos apps. Com uma chave nova, os apps já instalados **rejeitam** os próximos updates OTA, e é preciso um build nativo novo em todos os canais. Se a chave antiga estiver no gerenciador de senhas, pule este passo e use `OTA_KEY_FILE` apontando para ela.

1. Gere o par fora do repo:

```
mkdir -p ~/agenda-ota-keys && cd apps/mobile
npx expo-updates codesigning:generate \
  --key-output-directory ~/agenda-ota-keys \
  --certificate-output-directory ./certs \
  --certificate-validity-duration-years 10 \
  --certificate-common-name "Agenda de Boteco"
cp ./certs/certificate.pem ../mobile-client/certs/certificate.pem
cd ../..
```

2. Guarde `~/agenda-ota-keys/private-key.pem` no gerenciador de senhas e apague a pasta no fim do passo 10.
3. Commite os dois `certificate.pem` novos (é chave pública, pode ir para o git):

```
git add apps/mobile/certs/certificate.pem apps/mobile-client/certs/certificate.pem
git commit -m "Rotate OTA code signing certificate"
```

4. Carregar os valores no terminal

Cole o bloco abaixo, substitua cada `COLE_AQUI` pelo valor do passo 2 e pressione Enter. Os passos 5 a 8 leem estas variáveis.

```
# --- Supabase ---
export SB_REF="COLE_AQUI"
export SB_URL="https://${SB_REF}.supabase.co"
export SB_ANON="COLE_AQUI"
export SB_SERVICE="COLE_AQUI"
export SB_DB_PASSWORD="COLE_AQUI"
export SB_ACCESS_TOKEN="COLE_AQUI"

# --- PostHog ---
export PH_INGEST_HOST="https://us.i.posthog.com"
export PH_API_HOST="https://us.posthog.com"
export PH_KEY_CONSUMIDOR="COLE_AQUI"
export PH_KEY_DONO="COLE_AQUI"
export PH_KEY_ARTISTAS="COLE_AQUI"
export PH_KEY_ADMIN="COLE_AQUI"
export PH_KEY_LANDING="COLE_AQUI"
export PH_UPLOAD_KEY="COLE_AQUI"

# --- Cloudflare R2 ---
export R2_ACCOUNT="COLE_AQUI"
export R2_KEY_ID="COLE_AQUI"
export R2_SECRET="COLE_AQUI"
export R2_BUCKET_NAME="agenda-ota"
export R2_PUBLIC_URL="COLE_AQUI"

# --- OTA ---
export OTA_KEY_FILE="$HOME/agenda-ota-keys/private-key.pem"
export OTA_MANIFEST_URL="${SB_URL}/functions/v1/ota-manifest"

# --- Google Maps (app consumidor) ---
export GMAPS_IOS="COLE_AQUI"
export GMAPS_ANDROID="COLE_AQUI"

# --- Expo e Apple ---
export EXPO_TOKEN_VALUE="COLE_AQUI"
export ASC_ID_CONSUMIDOR="COLE_AQUI"
export ASC_ID_DONO="COLE_AQUI"

# --- URLs públicas ---
export SITE_URL="https://agendadeboteco.com"
export WEB_URL="${SITE_URL}/app"
```

Confira que nada ficou vazio ou com `COLE_AQUI`. O comando abaixo só imprime os nomes, nunca os valores:

```
for v in SB_REF SB_ANON SB_SERVICE SB_DB_PASSWORD SB_ACCESS_TOKEN PH_KEY_CONSUMIDOR PH_KEY_DON
O PH_KEY_ARTISTAS PH_KEY_ADMIN PH_KEY_LANDING PH_UPLOAD_KEY R2_ACCOUNT R2_KEY_ID R2_SECRET R2_
PUBLIC_URL GMAPS_IOS GMAPS_ANDROID EXPO_TOKEN_VALUE ASC_ID_CONSUMIDOR ASC_ID_DONO; do
  val="${(P)v}"; { [ -z "$val" ] || [ "$val" = "COLE_AQUI" ]; } && echo "FALTA: $v"
done; [ -f "$OTA_KEY_FILE" ] || echo "FALTA: arquivo $OTA_KEY_FILE"; echo "checagem concluída"
```

---

5. Supabase — segredos das Edge Functions

As funções `ota-manifest` e `send-push` leem `SUPABASE_URL` e `SUPABASE_SERVICE_ROLE_KEY`, que o Supabase injeta sozinho. Cadastre só os 2 segredos do OTA:

```
supabase secrets set --project-ref "$SB_REF" \
  OTA_PUBLIC_BASE_URL="$R2_PUBLIC_URL" \
  OTA_PRIVATE_KEY="$(cat "$OTA_KEY_FILE")"
```

Publique as funções, se o projeto for novo:

```
supabase link --project-ref "$SB_REF"
supabase functions deploy ota-manifest --project-ref "$SB_REF"
supabase functions deploy send-push --project-ref "$SB_REF"
```

---

6. GitHub Actions — secrets e variables

Usados pelos workflows `deploy.yml` e `ota-gc.yml`.

```
REPO="InovacodeDev/agenda-de-boteco"

# Secrets
gh secret set EXPO_TOKEN                --repo "$REPO" --body "$EXPO_TOKEN_VALUE"
gh secret set SUPABASE_URL              --repo "$REPO" --body "$SB_URL"
gh secret set SUPABASE_SERVICE_ROLE_KEY --repo "$REPO" --body "$SB_SERVICE"
gh secret set SUPABASE_ACCESS_TOKEN     --repo "$REPO" --body "$SB_ACCESS_TOKEN"
gh secret set SUPABASE_DB_PASSWORD      --repo "$REPO" --body "$SB_DB_PASSWORD"
gh secret set R2_ACCOUNT_ID             --repo "$REPO" --body "$R2_ACCOUNT"
gh secret set R2_ACCESS_KEY_ID          --repo "$REPO" --body "$R2_KEY_ID"
gh secret set R2_SECRET_ACCESS_KEY      --repo "$REPO" --body "$R2_SECRET"
gh secret set POSTHOG_CLI_TOKEN         --repo "$REPO" --body "$PH_UPLOAD_KEY"

# Variables (identificadores públicos)
gh variable set SUPABASE_PROJECT_REF --repo "$REPO" --body "$SB_REF"
gh variable set R2_BUCKET            --repo "$REPO" --body "$R2_BUCKET_NAME"
```

```
gh variable set ASC_APP_ID           --repo "$REPO" --body "$ASC_ID_CONSUMIDOR"
gh variable set ASC_APP_ID_CLIENT    --repo "$REPO" --body "$ASC_ID_DONO"
```

> **CUIDADO**
> Não crie `POSTHOG_CLI_PROJECT_ID` nem `EAS_PROJECT_ID` no GitHub. O ID do PostHog de cada app mobile vem do EAS (o OTA roda dentro de `eas env:exec`), e o ID do projeto EAS fica fixo no `app.config.ts`.

---

7. Vercel — 5 projetos web

O caminho mais simples é o painel. Em cada projeto: **Settings → Environment Variables**. O formulário aceita colar um bloco no formato `.env` inteiro.

1. Rode o comando do projeto (ele imprime o bloco `.env` já preenchido com os valores do passo 4).
2. Copie a saída e cole no campo *Key* da Vercel. Os campos se dividem sozinhos.
3. Marque **Production**, **Preview** e **Development** e salve.
4. Depois cadastre as 2 variáveis de upload **só em Production**, marcadas como *Sensitive* (bloco "Só Production" de cada projeto).
5. Não crie `NEXT_PUBLIC_VERCEL_ENV`, `VERCEL_ENV` nem `VERCEL_GIT_COMMIT_SHA`. A Vercel preenche essas.

> **ATENÇÃO**
> A saída dos comandos abaixo contém valores reais. Não salve em arquivo, não cole em chat. Depois de colar na Vercel, limpe a tela com `clear`.

7.1 `web` (Consumidor — projeto PostHog 591092)

```
printf 'NEXT_PUBLIC_SUPABASE_URL=%s\nNEXT_PUBLIC_SUPABASE_ANON_KEY=%s\nNEXT_PUBLIC_POSTHOG_KEY
=%s\nNEXT_PUBLIC_POSTHOG_HOST=%s\n' \
  "$SB_URL" "$SB_ANON" "$PH_KEY_CONSUMIDOR" "$PH_INGEST_HOST"
```

Só Production (Sensitive):

```
printf 'POSTHOG_PERSONAL_API_KEY=%s\nPOSTHOG_PROJECT_ID=591092\n' "$PH_UPLOAD_KEY"
```

7.2 `web-client` (Dono do bar — projeto PostHog 591110)

```
printf 'NEXT_PUBLIC_SUPABASE_URL=%s\nNEXT_PUBLIC_SUPABASE_ANON_KEY=%s\nNEXT_PUBLIC_POSTHOG_KEY
=%s\nNEXT_PUBLIC_POSTHOG_HOST=%s\n' \
  "$SB_URL" "$SB_ANON" "$PH_KEY_DONO" "$PH_INGEST_HOST"
```

Só Production (Sensitive):

```
printf 'POSTHOG_PERSONAL_API_KEY=%s\nPOSTHOG_PROJECT_ID=591110\n' "$PH_UPLOAD_KEY"
```

7.3 `web-artists` (Artistas — projeto PostHog 591111)

```
printf 'NEXT_PUBLIC_SUPABASE_URL=%s\nNEXT_PUBLIC_SUPABASE_ANON_KEY=%s\nNEXT_PUBLIC_SITE_URL=%s
\nNEXT_PUBLIC_POSTHOG_KEY=%s\nNEXT_PUBLIC_POSTHOG_HOST=%s\n' \
  "$SB_URL" "$SB_ANON" "$SITE_URL" "$PH_KEY_ARTISTAS" "$PH_INGEST_HOST"
```

Só Production (Sensitive):

```
printf 'POSTHOG_PERSONAL_API_KEY=%s\nPOSTHOG_PROJECT_ID=591111\n' "$PH_UPLOAD_KEY"
```

7.4 `admin` (Administrador — projeto PostHog 591113)

```
printf 'NEXT_PUBLIC_SUPABASE_URL=%s\nNEXT_PUBLIC_SUPABASE_ANON_KEY=%s\nNEXT_PUBLIC_POSTHOG_KEY
=%s\nNEXT_PUBLIC_POSTHOG_HOST=%s\n' \
  "$SB_URL" "$SB_ANON" "$PH_KEY_ADMIN" "$PH_INGEST_HOST"
```

Só Production (Sensitive):

```
printf 'POSTHOG_PERSONAL_API_KEY=%s\nPOSTHOG_PROJECT_ID=591113\n' "$PH_UPLOAD_KEY"
```

> **CUIDADO**
> O build do admin **falha** na Vercel sem `NEXT_PUBLIC_SUPABASE_URL`, porque a CSP precisa dessa origem. O mesmo vale para web-client e web-artists.

7.5 `landing` (Landing — projeto PostHog 591114)

A landing não usa Supabase. Os rewrites de `/app`, `/client`, `/admin` e `/artists` estão no `apps/landing/vercel.json`, com as URLs `*.vercel.app` fixas no arquivo.

```
printf 'NEXT_PUBLIC_SITE_URL=%s\nNEXT_PUBLIC_WEB_URL=%s\nNEXT_PUBLIC_ADMIN_URL=%s\nNEXT_PUBLIC
_WEB_CLIENT_URL=%s\nNEXT_PUBLIC_ARTISTS_URL=%s\nNEXT_PUBLIC_POSTHOG_KEY=%s\nNEXT_PUBLIC_POSTHO
G_HOST=%s\n' \
  "$SITE_URL" "$SITE_URL/app" "$SITE_URL/admin" "$SITE_URL/client" "$SITE_URL/artists" "$PH_KE
Y_LANDING" "$PH_INGEST_HOST"
```

Só Production (Sensitive):

```
printf 'POSTHOG_PERSONAL_API_KEY=%s\nPOSTHOG_PROJECT_ID=591114\n' "$PH_UPLOAD_KEY"
```

7.6 Redeploy

Variável nova só vale no próximo deploy. Em cada projeto: **Deployments → último deploy de Production → ⋯ → Redeploy** (desmarque o cache de build).

---

8. EAS — 2 apps mobile

Cada app tem 3 environments: `development`, `preview` (canais `alfa` e `beta`) e `production` (lojas). Visibilidade: `plaintext` para `EXPO_PUBLIC_*`, que é embutida no app; `sensitive` para chaves de mapa; `secret` para chave de upload.

Primeiro, defina a função auxiliar (só existe neste terminal):

```
easenv() { # uso: easenv <visibilidade> <NOME> <valor> <environments...>
  local vis="$1" name="$2" value="$3"; shift 3
  for env in "$@"; do
    eas env:create --environment "$env" --name "$name" --value "$value" \
      --visibility "$vis" --non-interactive --force >/dev/null \
      && echo "ok  $name ($env)" || echo "ERRO $name ($env)"
  done
}
```

8.1 `apps/mobile` (Consumidor — PostHog 591092)

```
cd apps/mobile
easenv plaintext EXPO_PUBLIC_SUPABASE_URL       "$SB_URL"            development preview produ
ction
easenv plaintext EXPO_PUBLIC_SUPABASE_ANON_KEY  "$SB_ANON"           development preview produ
ction
easenv plaintext EXPO_PUBLIC_POSTHOG_KEY        "$PH_KEY_CONSUMIDOR" development preview produ
ction
easenv plaintext EXPO_PUBLIC_POSTHOG_HOST       "$PH_INGEST_HOST"    development preview produ
ction
easenv plaintext EXPO_PUBLIC_SHARE_BASE_URL     "$SITE_URL"          development preview produ
ction
easenv plaintext EXPO_PUBLIC_OTA_MANIFEST_URL   "$OTA_MANIFEST_URL"  preview production
easenv sensitive GOOGLE_MAPS_API_KEY_IOS        "$GMAPS_IOS"         development preview produ
ction
easenv sensitive GOOGLE_MAPS_API_KEY_ANDROID    "$GMAPS_ANDROID"     development preview produ
ction
easenv secret    POSTHOG_CLI_API_KEY            "$PH_UPLOAD_KEY"     preview production
easenv plaintext POSTHOG_CLI_PROJECT_ID         "591092"             preview production
easenv plaintext POSTHOG_CLI_HOST               "$PH_API_HOST"       preview production
cd ../..
```

Sobre o EXPO_PUBLIC_SHARE_BASE_URL
Sem ele, o app gera deep links de scheme (`agenda-de-boteco://`). Com ele, gera URLs web. Os Universal/App Links ainda estão no backlog. Se ainda não quiser URL web, pule esta linha.

8.2 `apps/mobile-client` (Dono do bar — PostHog 591110)

```
cd apps/mobile-client
easenv plaintext EXPO_PUBLIC_SUPABASE_URL       "$SB_URL"           development preview produc
tion
easenv plaintext EXPO_PUBLIC_SUPABASE_ANON_KEY  "$SB_ANON"          development preview produc
tion
easenv plaintext EXPO_PUBLIC_POSTHOG_KEY        "$PH_KEY_DONO"      development preview produc
tion
easenv plaintext EXPO_PUBLIC_POSTHOG_HOST       "$PH_INGEST_HOST"   development preview produc
tion
easenv plaintext EXPO_PUBLIC_WEB_URL            "$WEB_URL"          development preview produc
tion
easenv plaintext EXPO_PUBLIC_OTA_MANIFEST_URL   "$OTA_MANIFEST_URL" preview production
easenv secret    POSTHOG_CLI_API_KEY            "$PH_UPLOAD_KEY"    preview production
easenv plaintext POSTHOG_CLI_PROJECT_ID         "591110"            preview production
easenv plaintext POSTHOG_CLI_HOST               "$PH_API_HOST"      preview production
cd ../..
```

CUIDADO
Nos perfis de loja (`preview` , `alfa` , `beta` , `production`), o `app.config.ts` **falha o build** sem `EXPO_PUBLIC_OTA_MANIFEST_URL` . No app consumidor, falha também sem as duas chaves do Google Maps.

8.3 Credenciais de assinatura

Keystore Android, certificados iOS e chave da App Store Connect não são variáveis de ambiente: ficam nos servidores do EAS. Em projeto novo, rode uma vez em cada app e siga o assistente:

```
cd apps/mobile && eas credentials && cd ../..
cd apps/mobile-client && eas credentials && cd ../..
```

---

9. Desenvolvimento local (`.env.local`)

Para rodar local contra o Supabase local (`supabase start`), os `.env.example` já trazem os valores padrão. Copie cada um:

```
for app in web web-client admin web-artists landing; do
  [ -f "apps/$app/.env.local" ] || cp "apps/$app/.env.example" "apps/$app/.env.local"
done
```

```
for app in mobile mobile-client; do
  [ -f "apps/$app/.env" ] || cp "apps/$app/.env.example" "apps/$app/.env"
done
```

1. Os comandos não sobrescrevem arquivo existente.
2. Abra cada arquivo no editor e preencha as chaves do PostHog **do projeto de dev**, se usar um. Sem key, o analytics vira no-op.
3. No mobile em aparelho físico, defina `EXPO_PUBLIC_DEV_HOST_IP` com o IP da sua máquina na rede local.
4. Nunca commite `.env` nem `.env.local`. O `.gitignore` já bloqueia.

---

10. Verificar tudo

Os comandos listam só nomes, sem valores.

```
gh secret list   --repo InovacodeDev/agenda-de-boteco
gh variable list --repo InovacodeDev/agenda-de-boteco
supabase secrets list --project-ref "$SB_REF"
(cd apps/mobile        && eas env:list --environment preview && eas env:list --environment pro
duction)
(cd apps/mobile-client && eas env:list --environment preview && eas env:list --environment pro
duction)
```

Teste do manifesto OTA. Espera HTTP `204` (nenhum update publicado) ou `200`. `500` com `misconfigured` indica falta de segredo do passo 5.

```
curl -s -w "\nHTTP %{http_code}\n" \
  -H "expo-protocol-version: 1" -H "expo-platform: android" \
  -H "expo-runtime-version: teste" -H "expo-channel-name: alfa" \
  -H "x-app-slug: agenda-de-boteco" "$OTA_MANIFEST_URL"
```

Teste do config mobile com o plugin de símbolos:

```
(cd apps/mobile-client && eas env:exec preview -- npx expo config --type public | grep -A3 pos
thog-react-native)
```

Ao terminar, limpe o terminal e apague a cópia local da chave do OTA (ela já está no gerenciador de senhas):

```
rm -rf ~/agenda-ota-keys
unset SB_SERVICE SB_DB_PASSWORD SB_ACCESS_TOKEN PH_UPLOAD_KEY R2_SECRET EXPO_TOKEN_VALUE GMAPS
_IOS GMAPS_ANDROID
clear
```

11. Provisionar o PostHog (script)

O script cria, em cada projeto: 9 insights, o dashboard `[Agenda] Saúde de erros`, 5 alertas, os destinos de Error Tracking (opcionais) e o Session Replay. Ele é idempotente. Rode primeiro em dry-run.

```
export POSTHOG_PERSONAL_API_KEY="COLE_A_PH_SETUP_KEY"
export POSTHOG_HOST="https://us.posthog.com"
# opcional: export POSTHOG_ALERT_WEBHOOK_URL="https://hooks.slack.com/..."
# opcional: export POSTHOG_ALERT_USER_EMAILS="voce@inovacode.dev"

for id in 591092 591110 591111 591113 591114; do POSTHOG_PROJECT_ID=$id pnpm posthog:setup; do
ne
```

Se o plano estiver certo, aplique:

```
for id in 591092 591110 591111 591113 591114; do POSTHOG_PROJECT_ID=$id pnpm posthog:setup --a
pply; done
unset POSTHOG_PERSONAL_API_KEY POSTHOG_ALERT_WEBHOOK_URL POSTHOG_ALERT_USER_EMAILS
```

12. Apêndice — inventário completo

12.1 Por destino

Variável	Onde	Quem usa	Tipo
`NEXT_PUBLIC_SUPABASE_URL`	Vercel: web, web-client, admin, web-artists	client Supabase + CSP	pública
`NEXT_PUBLIC_SUPABASE_ANON_KEY`	Vercel: web, web-client, admin, web-artists	client Supabase	pública
`NEXT_PUBLIC_POSTHOG_KEY`	Vercel: os 5	posthog-js	pública (por projeto)
`NEXT_PUBLIC_POSTHOG_HOST`	Vercel: os 5	rewrite `/ingest` + `ui_host`	pública
`NEXT_PUBLIC_SITE_URL`	Vercel: landing, web-artists	metadata/links	pública
`NEXT_PUBLIC_WEB_URL`, `_ADMIN_URL`, `_WEB_CLIENT_URL`, `_ARTISTS_URL`	Vercel: landing	links da landing	pública

`POSTHOG_PERSONAL_API_KEY`	Vercel: os 5, só Production	`@posthog/nextjs-config`	SECRET
`POSTHOG_PROJECT_ID`	Vercel: os 5, só Production	`@posthog/nextjs-config`	build
`EXPO_PUBLIC_SUPABASE_URL` / `_ANON_KEY`	EAS: os 2	client Supabase	pública
`EXPO_PUBLIC_POSTHOG_KEY` / `_HOST`	EAS: os 2	posthog-react-native	pública (por projeto)
`EXPO_PUBLIC_OTA_MANIFEST_URL`	EAS: os 2 (preview, production)	`expo-updates`	pública, obrigatória em loja
`EXPO_PUBLIC_SHARE_BASE_URL`	EAS: mobile	links de compartilhar	pública, opcional
`EXPO_PUBLIC_WEB_URL`	EAS: mobile-client	abrir página do evento	pública
`GOOGLE_MAPS_API_KEY_IOS` / `_ANDROID`	EAS: mobile	`react-native-maps`	sensitive, obrigatória em loja
`POSTHOG_CLI_API_KEY`	EAS: os 2 (preview, production)	upload de símbolos + OTA	SECRET
`POSTHOG_CLI_PROJECT_ID`	EAS: os 2 (591092 / 591110)	upload de símbolos + OTA	build
`POSTHOG_CLI_HOST`	EAS: os 2	upload de símbolos	build, opcional (padrão US)
`OTA_PUBLIC_BASE_URL`	Supabase secrets	Edge Function `ota-manifest`	pública
`OTA_PRIVATE_KEY`	Supabase secrets	assinatura do manifesto	SECRET
`EXPO_TOKEN`	GitHub secret	`deploy.yml` (EAS)	SECRET
`SUPABASE_URL` / `SUPABASE_SERVICE_ROLE_KEY`	GitHub secret	publish/prune do OTA	SECRET
`SUPABASE_ACCESS_TOKEN` / `SUPABASE_DB_PASSWORD`	GitHub secret	`supabase db push`	SECRET
`R2_ACCOUNT_ID` / `R2_ACCESS_KEY_ID` / `R2_SECRET_ACCESS_KEY`	GitHub secret	upload/GC do OTA	SECRET

`POSTHOG_CLI_TOKEN`	GitHub secret	alias da chave de upload no OTA	SECRET
`SUPABASE_PROJECT_REF`	GitHub variable	`supabase link`	pública
`R2_BUCKET`	GitHub variable	OTA (padrão `agenda-ota`)	pública
`ASC_APP_ID` / `ASC_APP_ID_CLIENT`	GitHub variable	submit iOS	pública

12.2 Automáticas — não criar

Variável	Quem preenche
`VERCEL`, `VERCEL_ENV`, `NEXT_PUBLIC_VERCEL_ENV`, `VERCEL_URL`, `VERCEL_GIT_COMMIT_SHA`	Vercel
`EAS_BUILD_PROFILE`	EAS Build
`SUPABASE_URL`, `SUPABASE_SERVICE_ROLE_KEY` (dentro das Edge Functions)	Supabase
`GITHUB_SHA`, `GITHUB_OUTPUT`, `RUNNER_TEMP`, `CI`	GitHub Actions
`NODE_ENV`	Next/Expo

12.3 Fora de variáveis de ambiente, mas necessário num projeto do zero

- **Supabase Auth** (painel → Authentication): provedores Google e Apple (client ID e secret), URLs de redirect (`/client/login`, `/app/...`, schemes `agenda-de-boteco://` e `agenda-boteco-client://`), SMTP e templates de e-mail (`supabase/emails/`).
- **Supabase banco**: `supabase db push` aplica as migrações (já roda no `deploy.yml` em `release`).
- **Cloudflare R2**: lifecycle do bucket com `node scripts/ota/setup-r2-lifecycle.mjs` (usa as mesmas `R2_*`).
- **EAS**: credenciais de assinatura (passo 8.3) e conta de serviço do Google Play para submit.
- **Vercel**: domínio da landing e os nomes `agenda-de-boteco-{web,admin,client,artists}.vercel.app` usados nos rewrites do `apps/landing/vercel.json`.